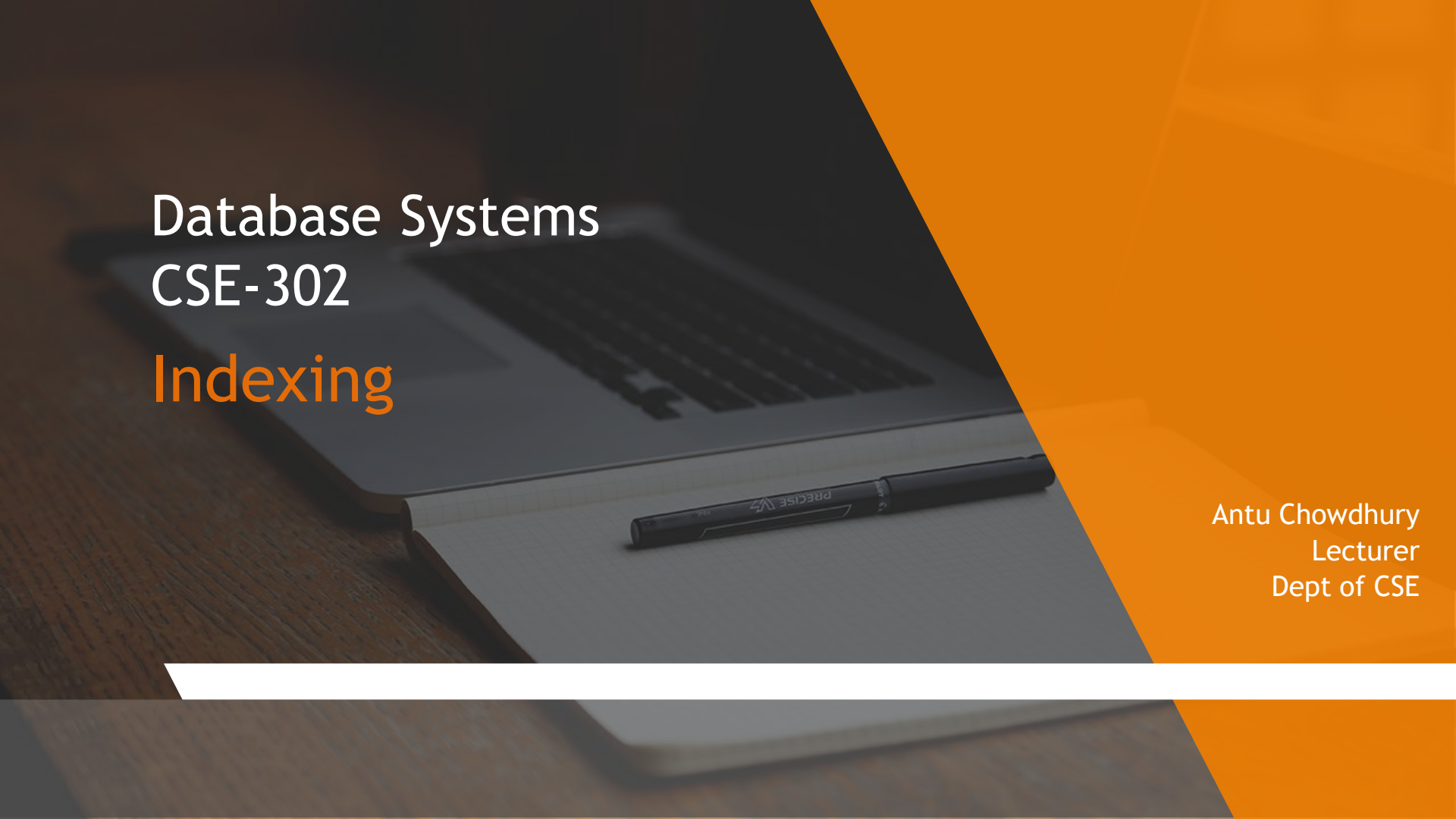




Database Systems

CSE-302

Indexing

Antu Chowdhury
Lecturer
Dept of CSE

Indexing

- Indexes are used to quickly locate data without having to search every row in a database every time.

Why Indexing

- To **reduce the number of disk accesses (I/O cost)** needed to find records.
 - **I/O cost** refers to the number of **disk read/write operations** (Input/output operations) the system performs during data retrieval or updates.
- DBMS stores information on some storage medium
 - **Primary storage:**
 - can be operated directly by CPU.
 - **Secondary storage:**
 - larger capacity, lower cost, slower access
 - cannot be operated directly by CPU
 - must be copied to primary storage

Why Not Store Everything in Main Memory?

- Cost and size
- Main memory is volatile: What's the problem? You know!!!

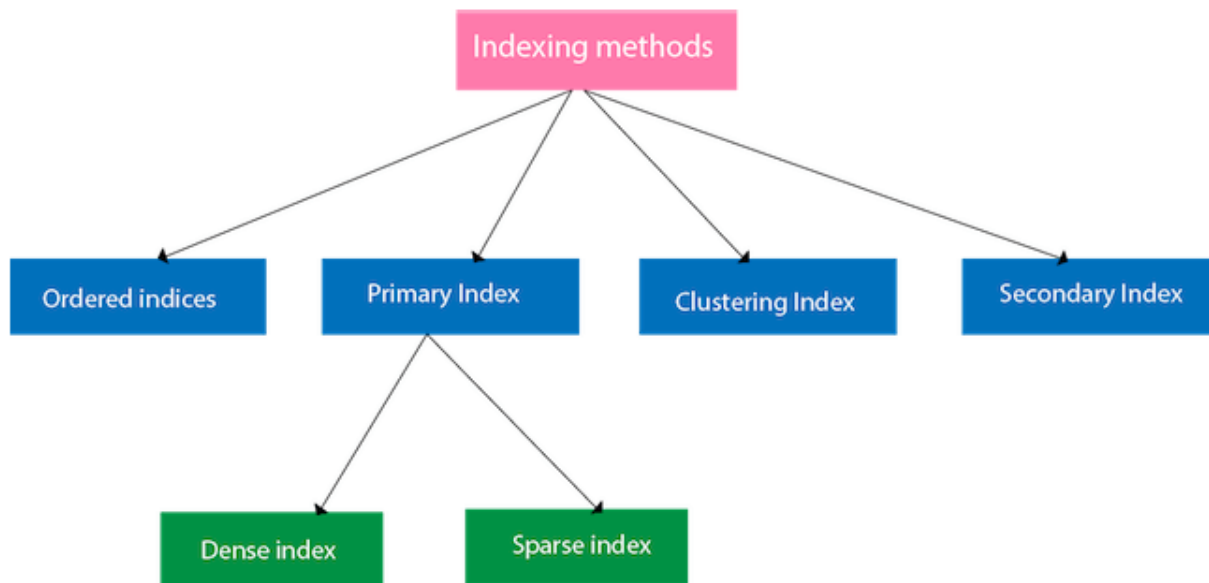
Indexing : Basic Concepts

- Indexing mechanisms used to speed up access to desired data.
 - E.g., author catalog in library
- **Search Key** – An attribute or, set of a attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form

search-key	pointer
------------	---------

- Index files are typically much smaller than the original file

Indexing Methods



Indexing Methods (Ordered Index)

- In an **ordered index**, index entries are stored in **sorted order** based on the **search key value**.

Indexing Methods-Primary Index

- If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record and contain 1:1 relation between the records.
- As primary keys are stored in sorted order, the performance of the searching operation is quite efficient.
- The primary index can be classified into two types:
- Dense index and Sparse index.

Indexing Methods-Dense Index

- **Dense index** — Index record appears for every search-key value in the file.
- In this, the number of records in the index table is same as the number of records in the main table.

10101		→	10101	Srinivasan	Comp. Sci.	65000	
12121		→	12121	Wu	Finance	90000	
15151		→	15151	Mozart	Music	40000	
22222		→	22222	Einstein	Physics	95000	
32343		→	32343	El Said	History	60000	
33456		→	33456	Gold	Physics	87000	
45565		→	45565	Katz	Comp. Sci.	75000	
58583		→	58583	Califieri	History	62000	
76543		→	76543	Singh	Finance	80000	
76766		→	76766	Crick	Biology	72000	
83821		→	83821	Brandt	Comp. Sci.	92000	
98345		→	98345	Kim	Elec. Eng.	80000	



Indexing Methods-Sparse Index

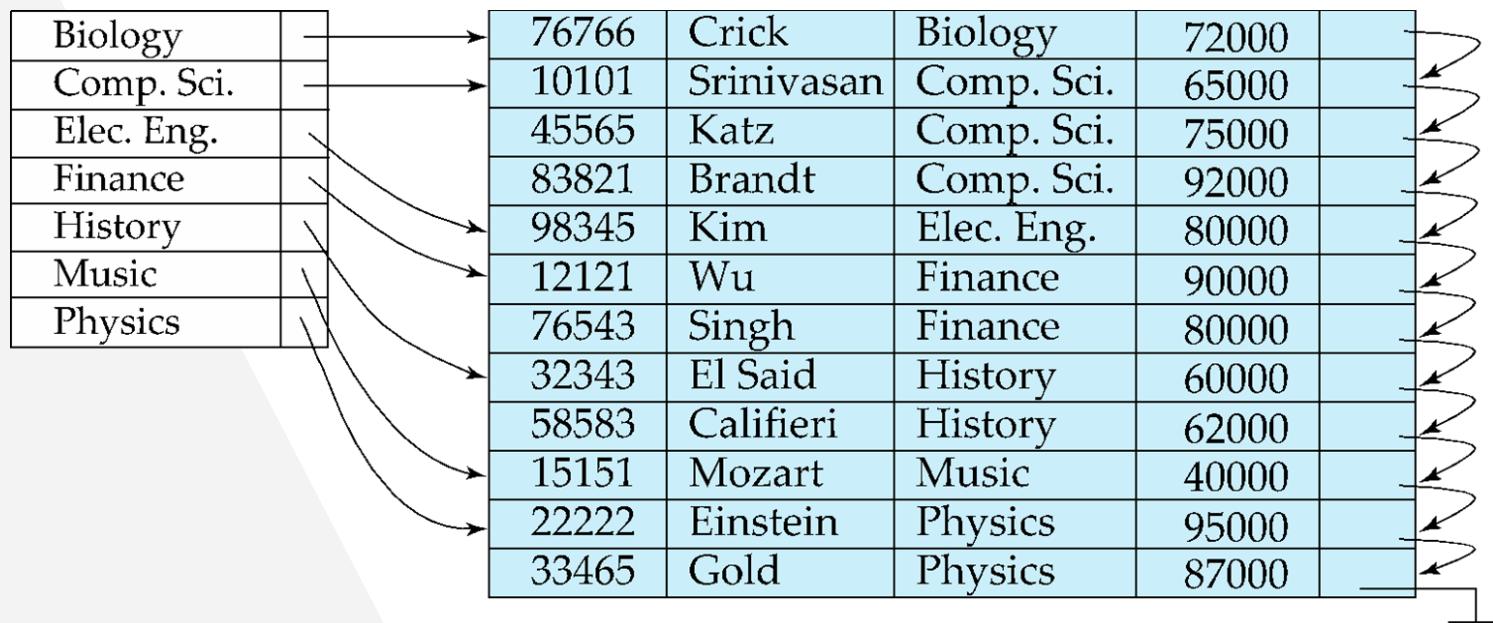
- **Sparse index** — contains index records for only some search-key values.
 - Applicable when records are sequentially ordered on search-key
- To locate a record with search-key value K we:
 - Find index record with largest search-key value $< K$
 - Search file sequentially starting at the record to which the index record points

10101		10101	Srinivasan	Comp. Sci.	65000	
32343		12121	Wu	Finance	90000	
76766		15151	Mozart	Music	40000	
		22222	Einstein	Physics	95000	
		32343	El Said	History	60000	
		33456	Gold	Physics	87000	
		45565	Katz	Comp. Sci.	75000	
		58583	Califieri	History	62000	
		76543	Singh	Finance	80000	
		76766	Crick	Biology	72000	
		83821	Brandt	Comp. Sci.	92000	
		98345	Kim	Elec. Eng.	80000	

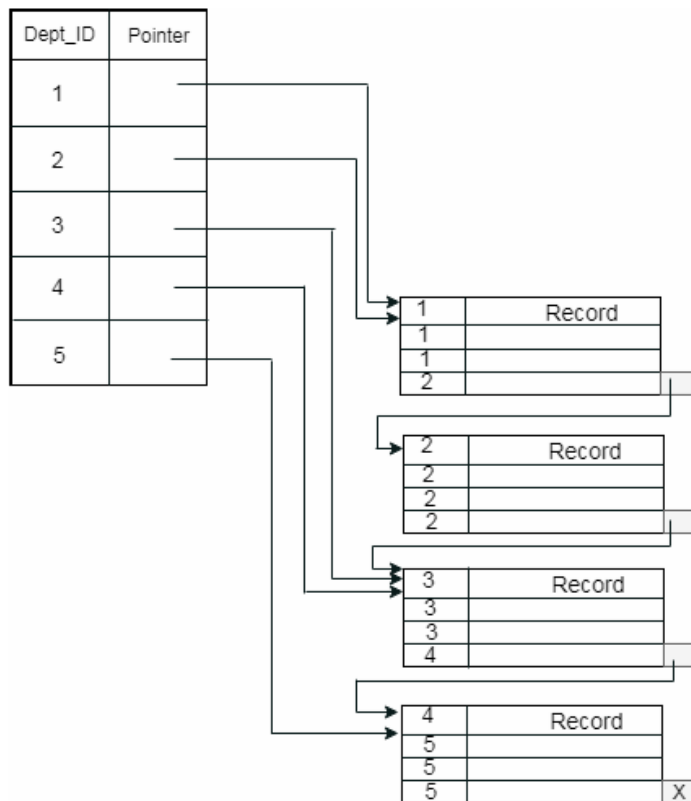
Indexing Methods-Clustering Index

- Clustering indexing is defined on a **ordered** data file. The data file is sorted on a non-key field like age of a student or salary of an employee etc. In real world, there are cases where we want to search on the basis of non unique key. For such cases, the index is created on non-primary key columns which may not be unique for each record.
- In such cases, in order to identify the records faster, we will group two or more columns together to get the unique values and create index out of them. This method is known as the clustered indexing
- There will be only one entry for each unique value of non key attribute in the index table.
- if all records with the same key value **cannot fit into a single block**, the remaining records are stored in **subsequent blocks**. To maintain continuity, the database uses **block-level pointers to link these blocks together**.

Indexing Methods-Clustering Index



Indexing Methods-Clustering Index

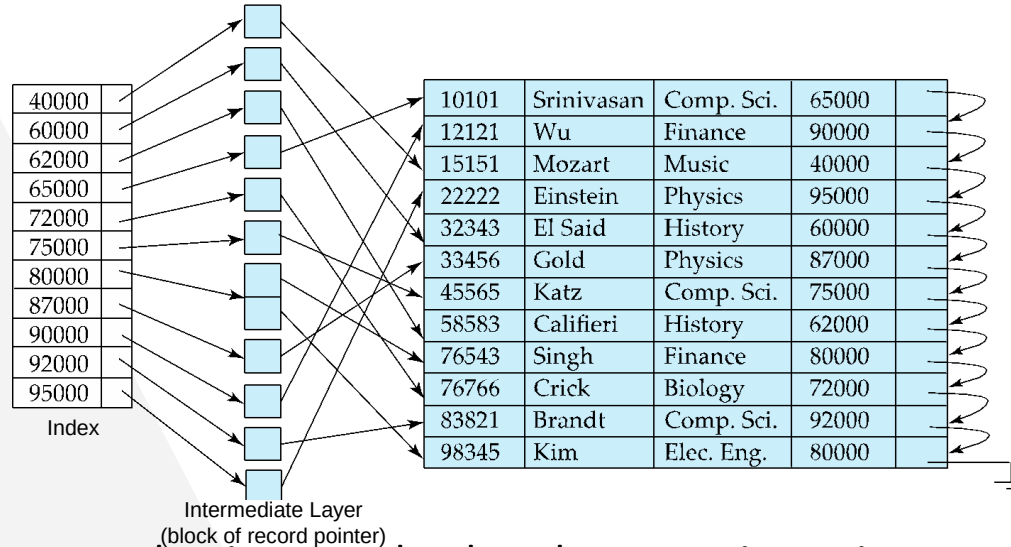


Indexing Methods-Secondary Index

- A **secondary index** is an index built on a **non-primary key attribute** (i.e., a column that is not the main key ordering the data).
- Data Storage Order: Data records are **not physically ordered** by the secondary index key. The table remains ordered by the primary key (or original order).
- Index Structure Order: The secondary index itself is **ordered by the secondary key**, but the data file is not reordered.

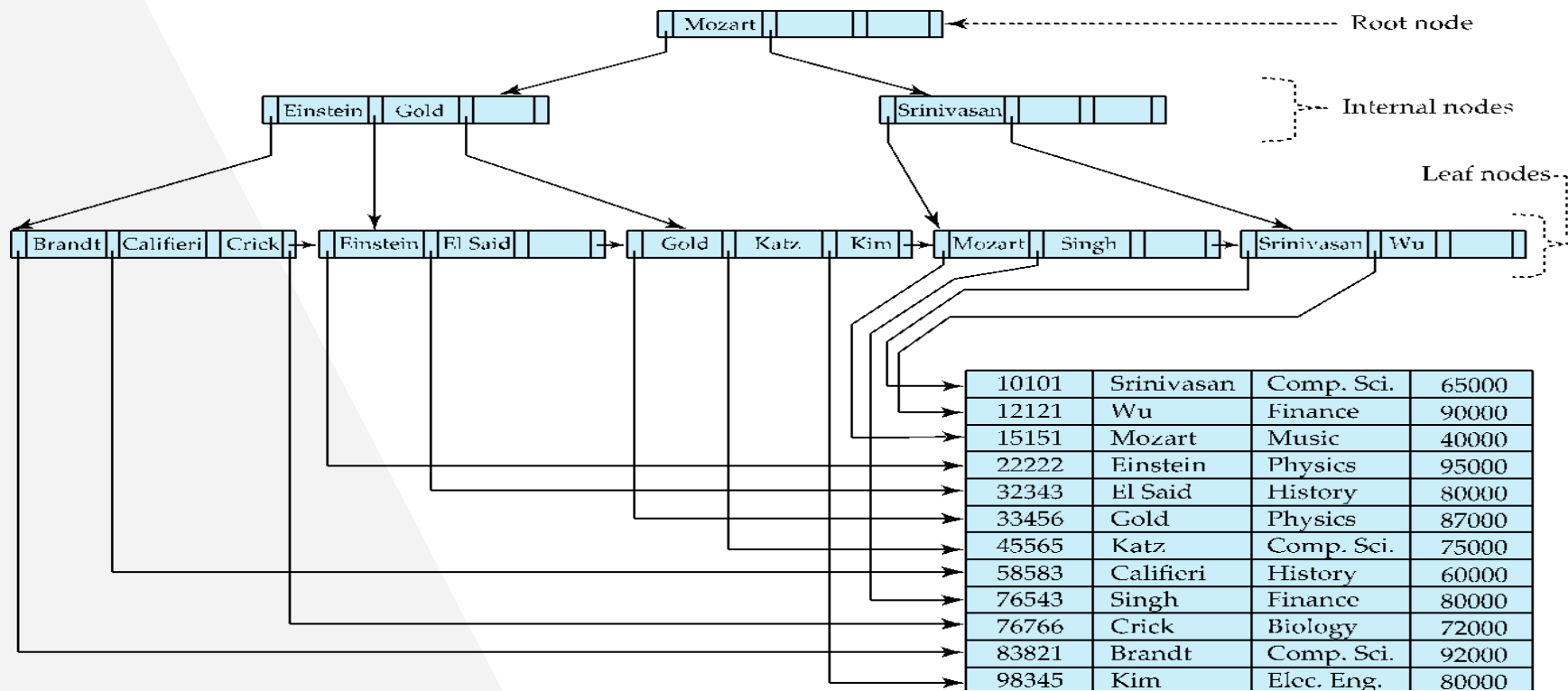
Indexing Methods-Secondary Index

- Secondary index on salary field of instructor

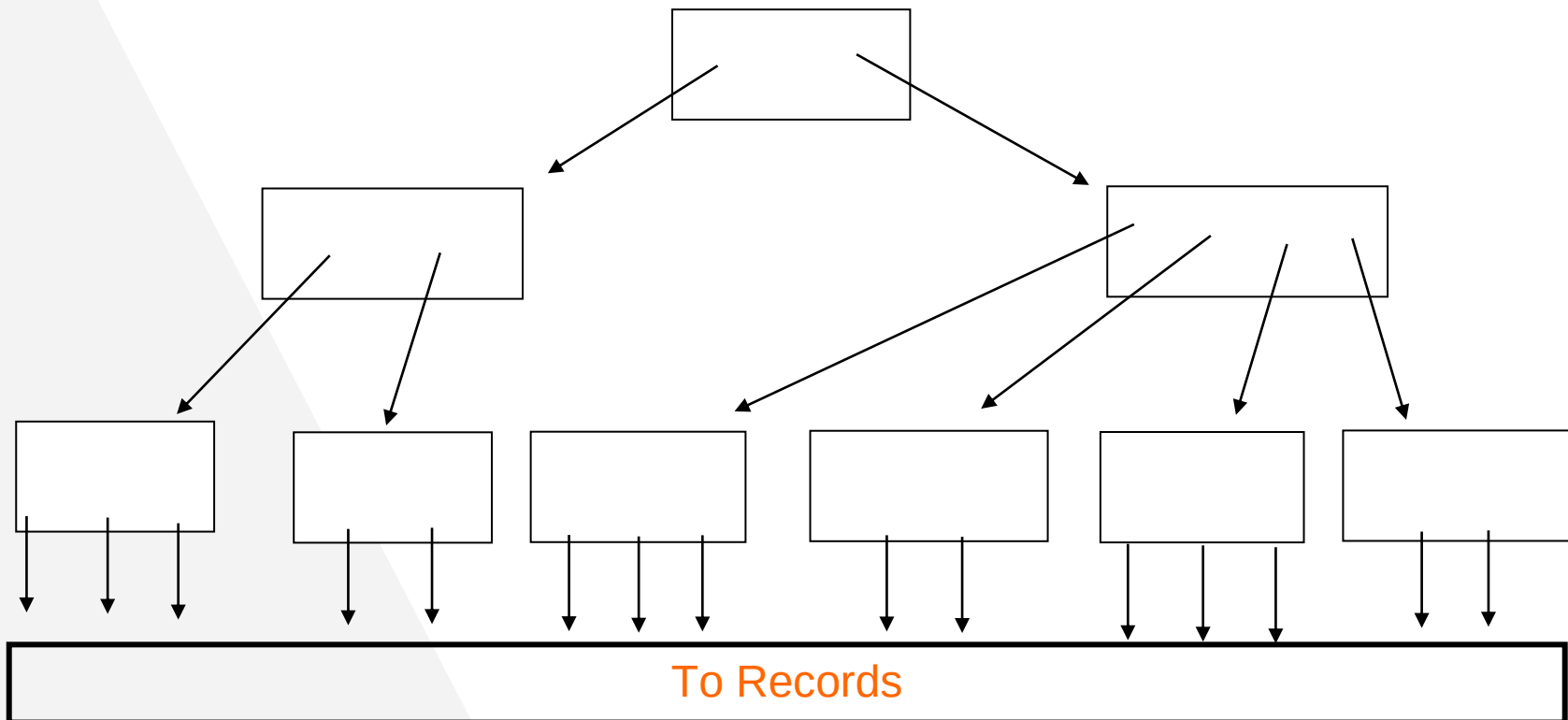


- Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.

B+ Tree Index Files



B+ Tree Index Files



B+ Tree Index Files

B⁺-tree is a rooted tree satisfying the following properties

- B+ Tree, values (or keys) must be in sorted order
- All paths from root to leaf are of the same length
- Each node that is not a root or a leaf has between $\lceil n/2 \rceil$ and n children.
- A leaf node has between $\lceil (n-1)/2 \rceil$ and $n-1$ values
- Special cases:
 - If the root is not a leaf, it has at least 2 children.
 - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n-1)$ values.

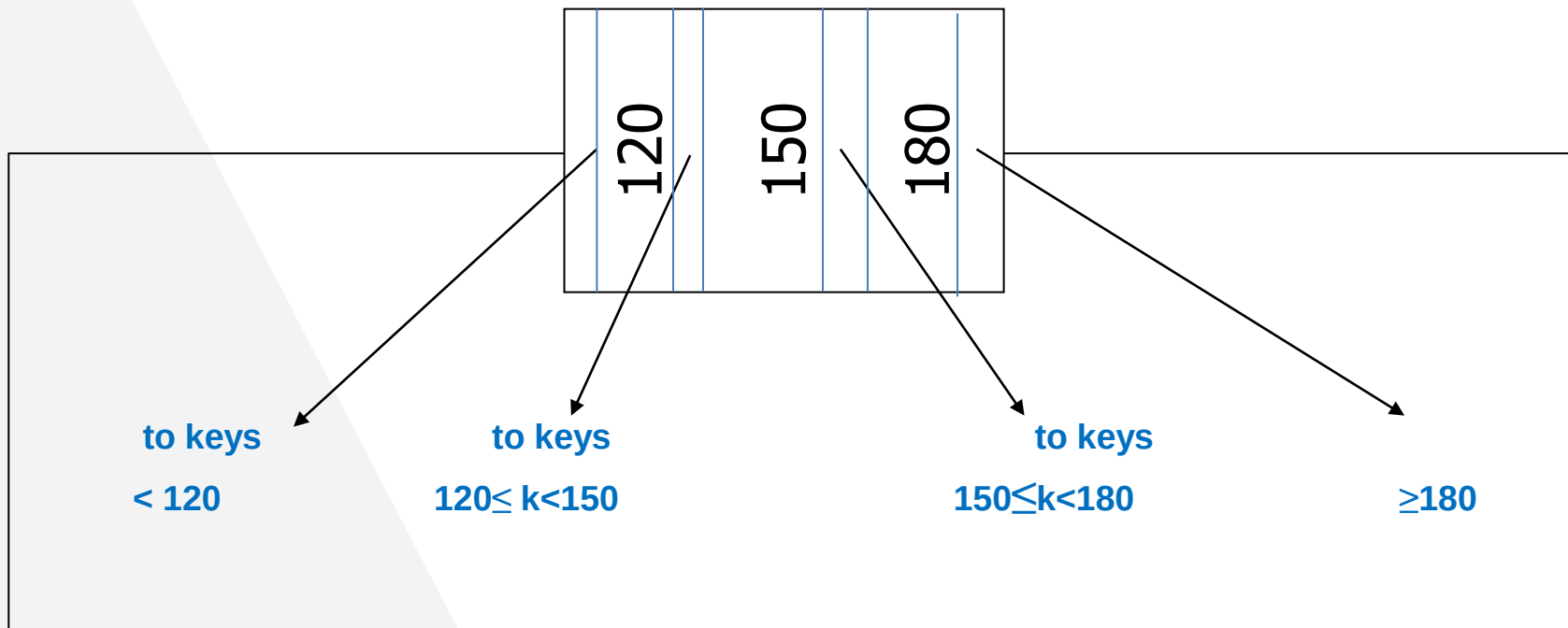
B+ Tree Index Files

- Typical node
 - K_i are the search-key values
 - P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).
- The search-keys in a node are ordered

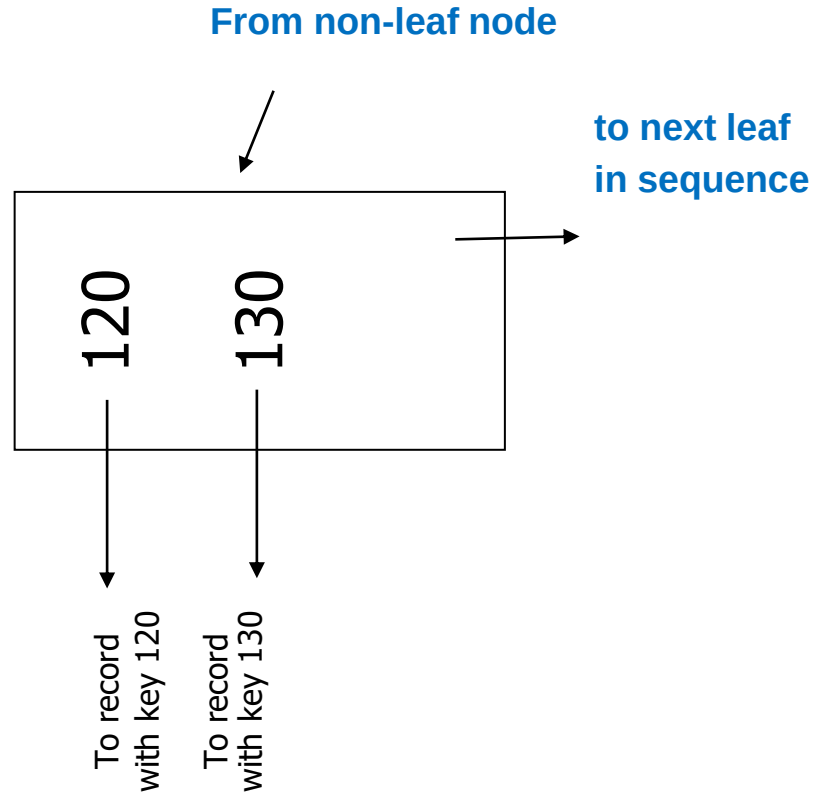
$$K_1 < K_2 < K_3 < \dots < K_{n-1}$$



B+ Tree Index Files-Non leaf Node

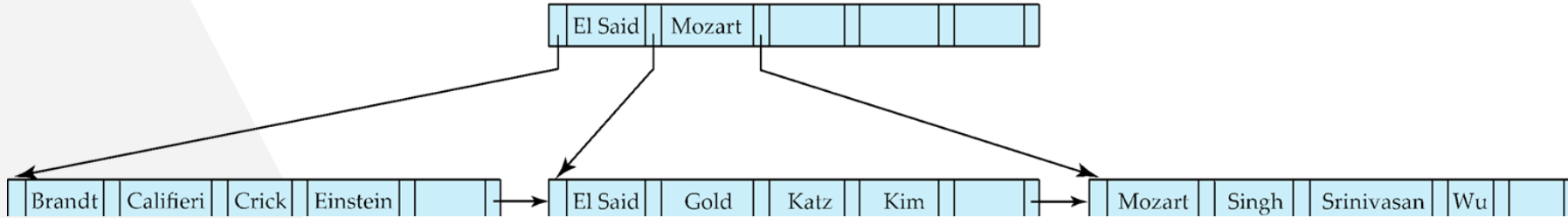


B+ Tree Index Files- leaf Node



B+ Tree Index Files- leaf Node

- B⁺-tree for *instructor* file ($n = 6$)



- Leaf nodes must have between 3 and 5 values ($\lceil (n-1)/2 \rceil$ and $n-1$, with $n = 6$).
- Non-leaf nodes other than root must have between 3 and 6 children ($\lceil n/2 \rceil$ and n with $n = 6$).
- Root must have at least 2 children.

B+ Tree Creation

Order, $n = 4$

Non leaf Max children = 4 , Non leaf min children = $\text{ceil}(n/2) = 2$

Leaf node Max key = $(n-1)=3$, Leaf node min key = $\text{ceil}((n-1)/2) = 2$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Insert 1



Insert 4



Insert 7



B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Insert 10



Find the middle key to split.

Here 4 and 7.

but which one should we take?

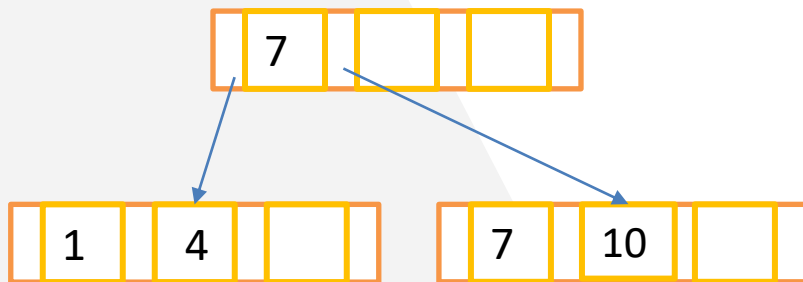
- If 4, then the tree will be left biased. And for all split we need to split like this.
- If 7, then the tree will be right biased

B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

After split



**data is always to be inserted in leaf node

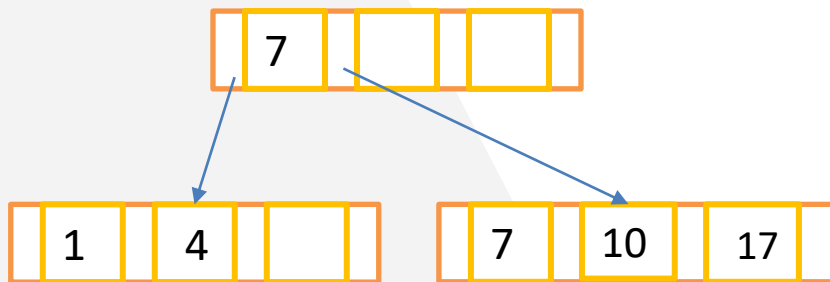
**tree grows towards root

B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Insert 17



**data is always to be inserted in leaf node

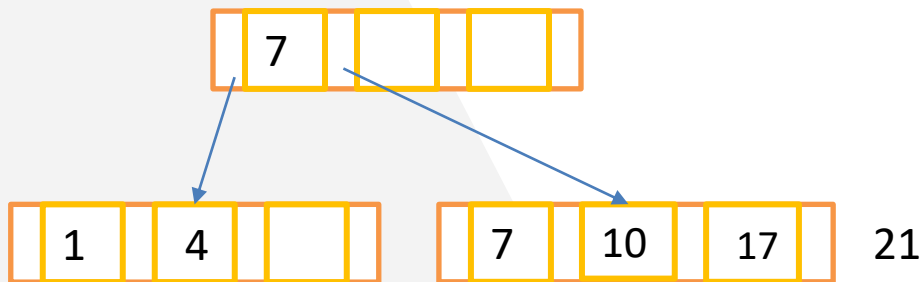
**tree grows towards root

B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Insert 21



**data is always to be inserted in leaf node

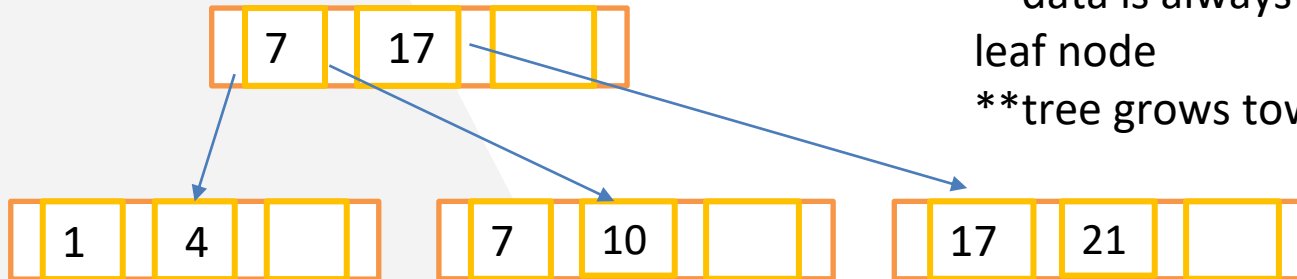
**tree grows towards root

B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Split at 17



**data is always to be inserted in leaf node

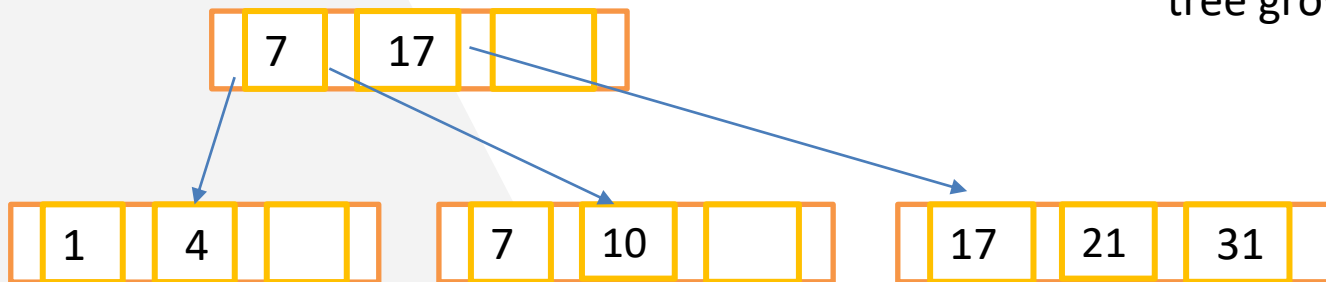
**tree grows towards root

B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Insert 31



**data is always to be inserted in leaf node

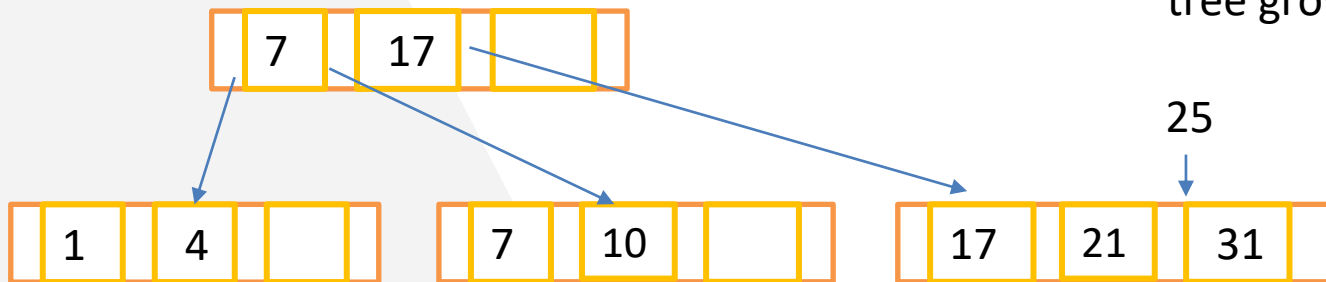
**tree grows towards root

B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Insert 25



**data is always to be inserted in leaf node

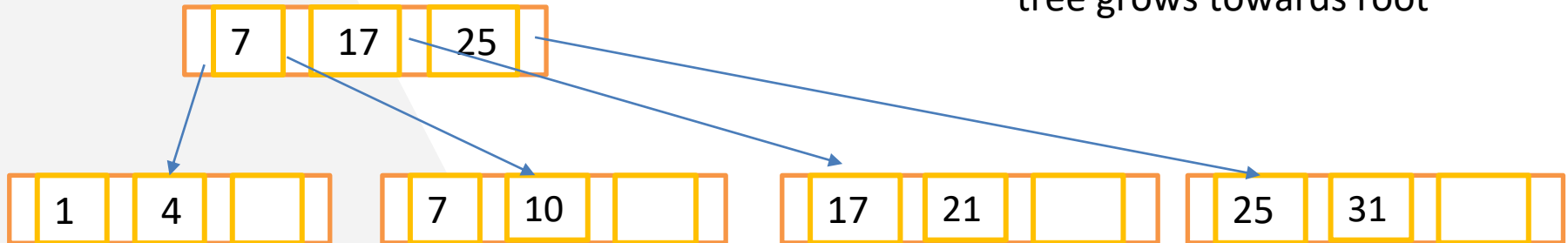
**tree grows towards root

B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Split at 25



**data is always to be inserted in leaf node

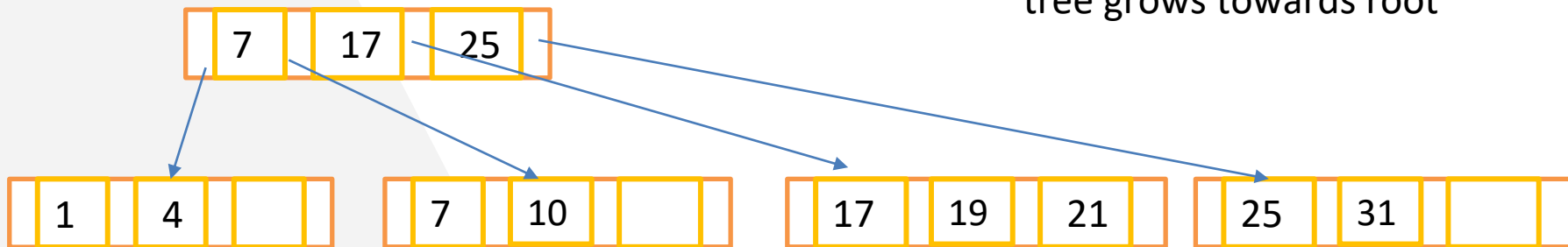
**tree grows towards root

B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Insert 19



**data is always to be inserted in leaf node

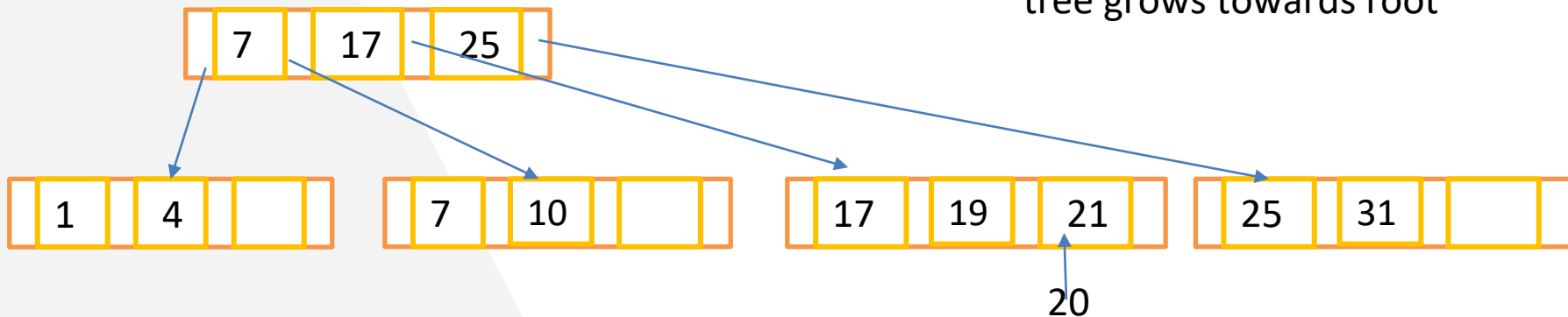
**tree grows towards root

B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Insert 20

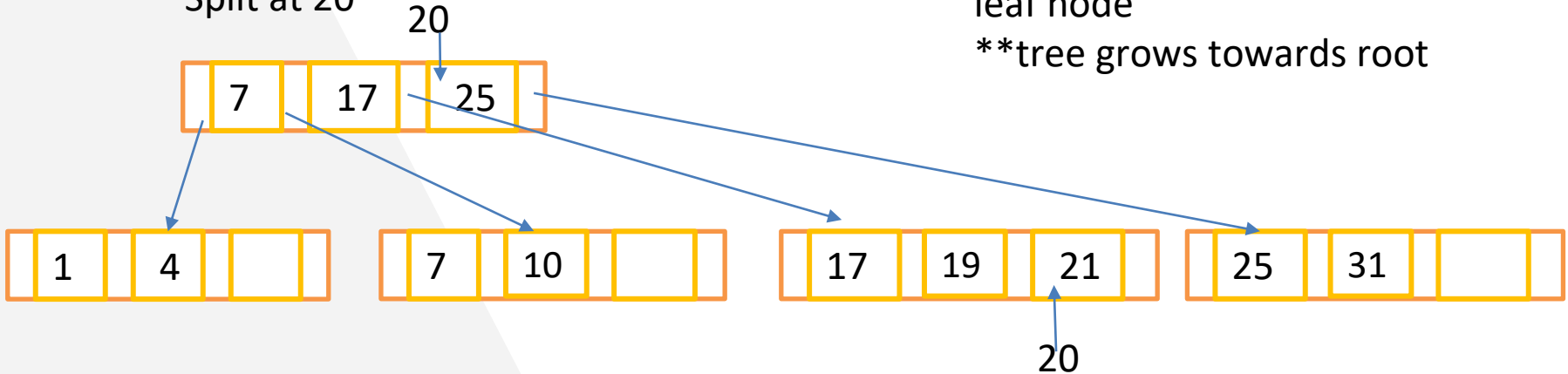


B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Split at 20



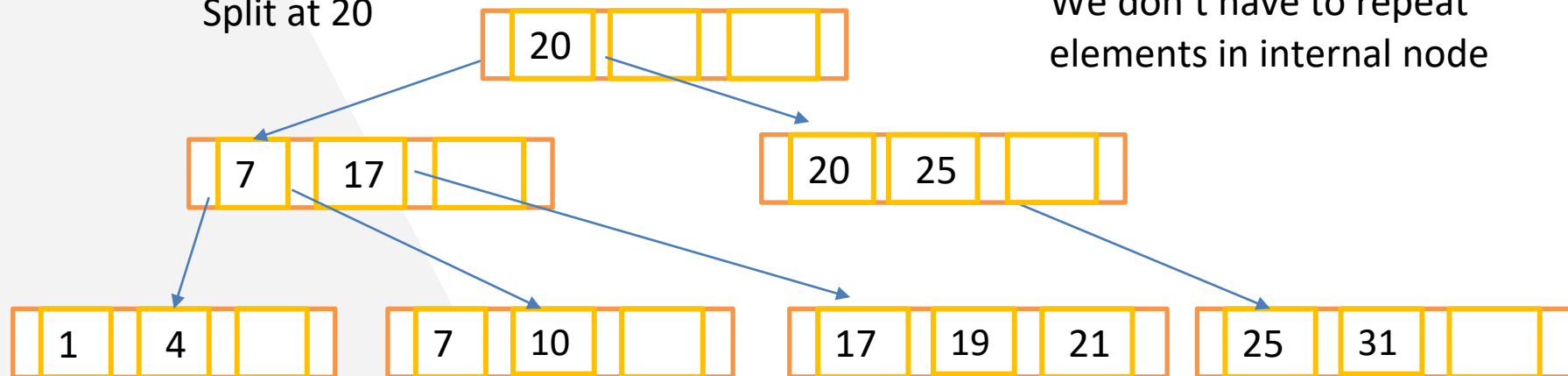
B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42



Split at 20

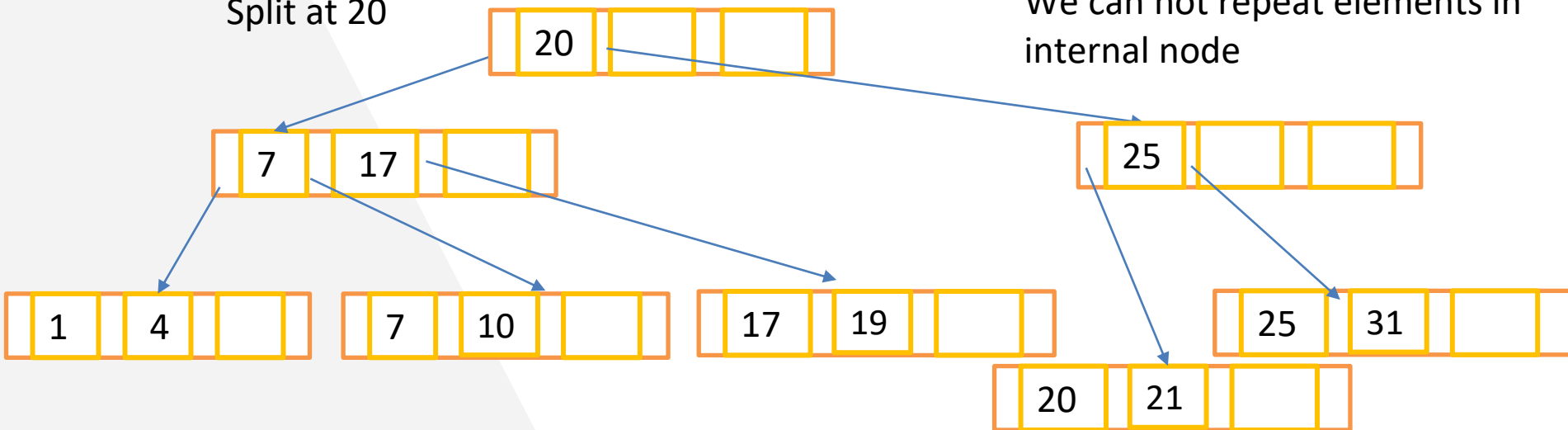


B+ Tree Creation

Order, $n = 4$

Data: 1,4,7,10,17,21,31,25,19,20,28,42

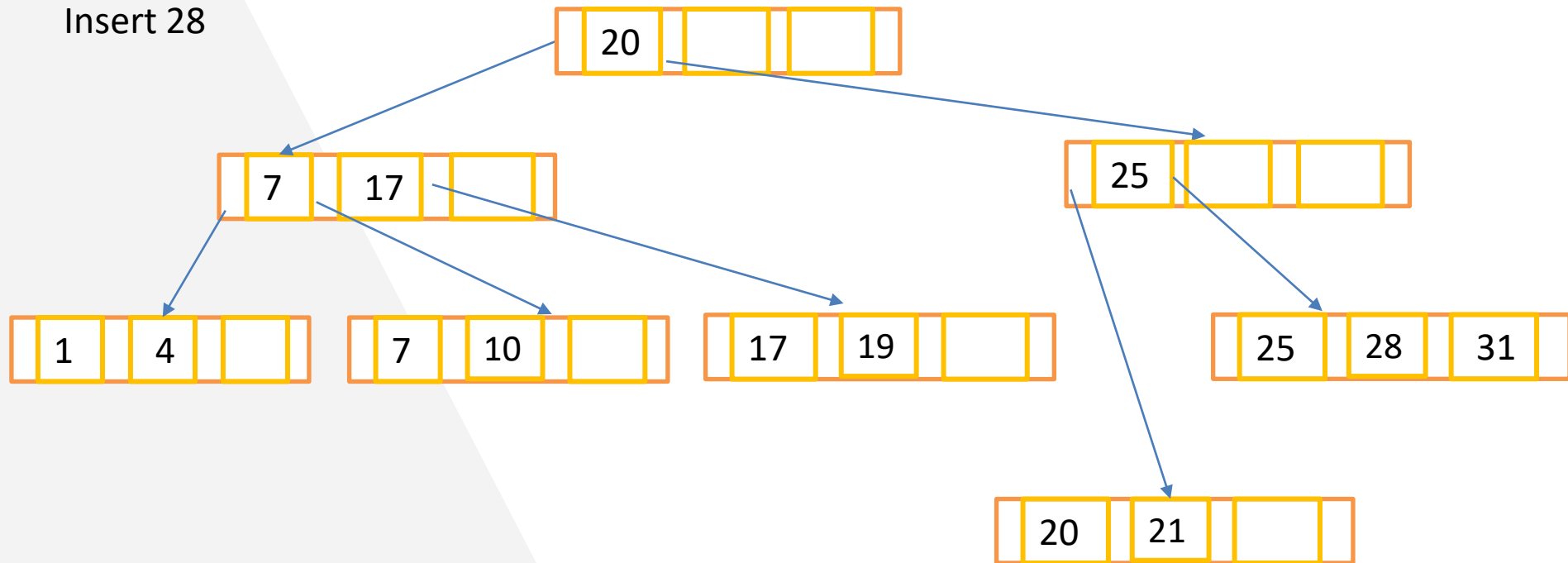
Split at 20



B+ Tree Creation

Data: 1,4,7,10,17,21,31,25,19,20,28,42

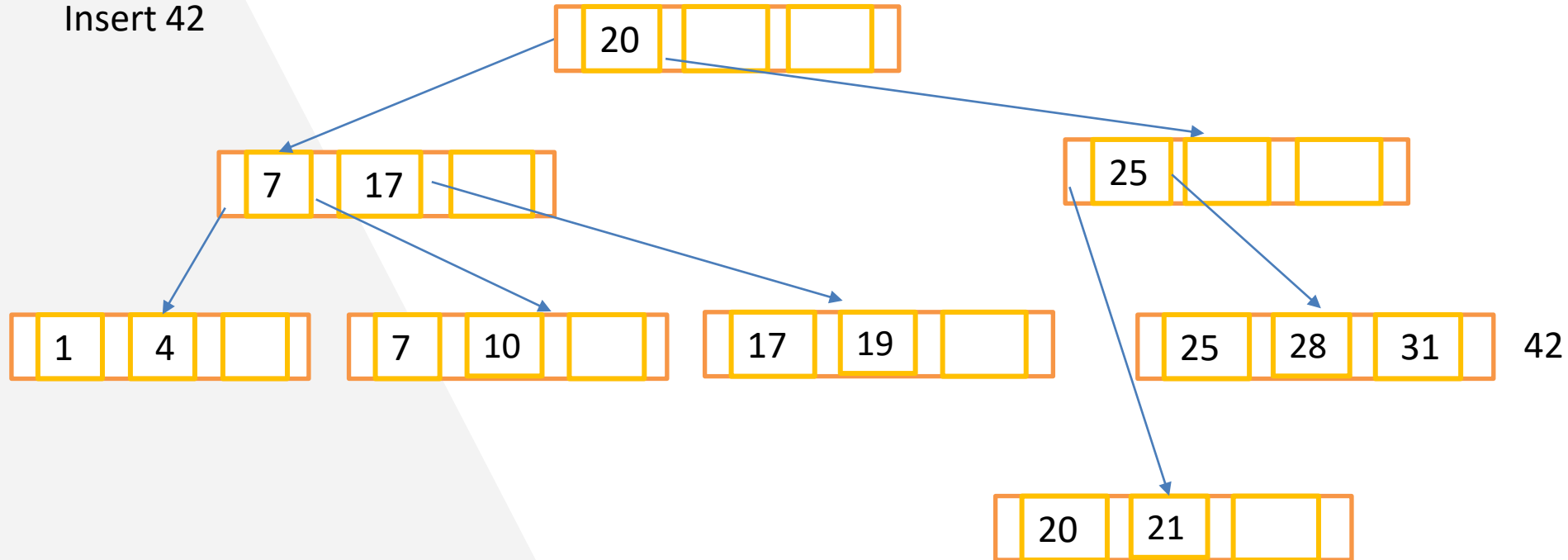
Insert 28



B+ Tree Creation

Data: 1,4,7,10,17,21,31,25,19,20,28,42

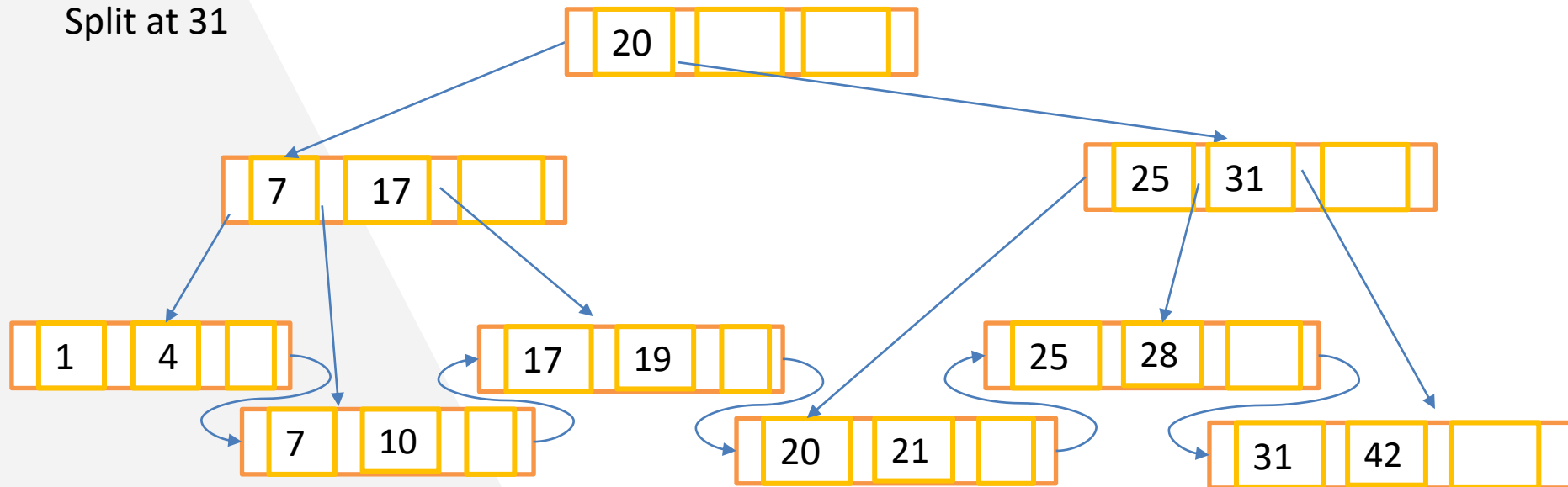
Insert 42



B+ Tree Creation

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Split at 31



B+ Tree Creation

Data: 1,4,7,10,17,21,31,25,19,20,28,42

Split at 31

